Unit - II

Working with Data in Python

Contents:

A file is a named location used for storing data.

For example, `main.py` is a file that is always used to store Python code.

File Operations in Python

File operations are an essential part of programming, allowing data to be stored and retrieved from files on a computer.

Python provides built-in functions to handle file operations, including reading, writing, appending, and deleting files, a process known as file handling.

- Python treats files as text or binary.

- Each line of a file is terminated with a special character, called the EOL or End of Line characters like comma {,} or newline character {\n}.

Types of File Operations

Python supports various file operations:

1. **Opening a File (**open()**)**
2. **Reading from a File (**read()**,** readline()**,** readlines()**)**
3. **Writing to a File (**write()**,** writelines()**)**
4. **Appending to a File (**append **mode)**
5. **Closing a File (**close()**)**
6. **Deleting a File (**os.remove()**)**

File operations in Python allow you to create, read, write, and manipulate files. Below are common file operations with examples and their respective outputs.

1. Opening a File

In python we need to open a file before performing any operations on it.

Python provides the **open()** function to open files.

Syntax:

```
file = open("filename", mode)
```

Common modes :

- "r": Read (default)
- "w": Write (creates or overwrites)
- "a": Append
- "x": Create (fails if file exists)
- "r+": Read & Write
- w+: To write and read data. It will override existing data.
- a+: To append and read data from the file. It won't override existing data.

Suppose we have a file named as example.txt. To perform any operations on example.txt we need to open it first with open() function.
Example:

```
file = open("example.txt")
```

- default mode is read if not mentioned any mode while opening a file.

Different ways of opening a file:

Using open () function

Using full path of a file include in open() (open("home/documents/example.txt")

Using with .. open ()

2. Writing to a File

```
with open("example.txt", "w") as file:
  file.write("Hello, World!\n")
```

```
    file.write("This is a test file.")
print("Data written successfully!")
```

Output:

```
Data written successfully!
```

Contents of example.txt**:**

```
Hello, World!
This is a test file.
```

3. Reading from a File

a) Read Entire File

```
with open("example.txt", "r") as file:
    content = file.read()

print(content)
```

Output:

```
Hello, World!
This is a test file.
```

b) Read Line by Line

```
with open("example.txt", "r") as file:
    for line in file:
        print(line.strip())  # strip() removes extra newline
```

Output:

```
Hello, World!
This is a test file.
```

4. Appending to a File

```
with open("example.txt", "a") as file:
    file.write("\nAppending new line.")

print("Data appended successfully!")
```

Output:

Data appended successfully!

Contents of example.txt **after appending:**

```
Hello, World!
This is a test file.
Appending new line.
```

5. Reading File into a List

```
with open("example.txt", "r") as file:
    lines = file.readlines()

print(lines)  # List of lines
```

Output:

```
['Hello, World!\n', 'This is a test file.\n', 'Appending new line.\n']
```

6. Checking if a File Exists (Before Deleting)

```
import os

if os.path.exists("example.txt"):
    print("File exists")
else:
    print("File does not exist")
```

Output:

File exists

7. Deleting a File

```
import os

if os.path.exists("example.txt"):
    os.remove("example.txt")
    print("File deleted successfully!")
else:
    print("File does not exist.")
```

Output:

```
File deleted successfully!
```

8. Working with Binary Files (Images, PDFs)

```
with open("sample.jpg", "rb") as file:
    data = file.read()
    print(f"Read {len(data)} bytes")
```

Output (example for an image file):

```
Read 1048576 bytes
```

9. Using with Statement (Best Practice)

Using with open(...) ensures the file is closed automatically after the operation.

```
with open("data.txt", "w") as file:
    file.write("Safe file handling with 'with' statement.")
```

Python provides efficient file handling through various modes and functions. Using the with statement ensures safe and automatic file closure.
File operations are fundamental for data storage, logging, and configuration management in applications

Examples

Read line by line

```
file = open('geek.txt', 'r')
# This will print every line one by one in the file
for each in file:
    print (each)
```

Read the first five characters of stored data and return it as a string:

```
file = open("file.txt", "r")
print (file.read(5))
```

Let’s see how to create a file and how to write mode works

```
file = open('geek.txt','w')
file.write("This is the write command")
file.write("It allows us to write in a particular file")
file.close()
```

```
# Python code to illustrate append() mode

file = open('geek.txt', 'a')
file.write("This will add this line")
file.close()
```

Using 'with' , this method any files opened will be closed automatically after one is done, so auto-cleanup

```
# Python code to illustrate with()

with open("file.txt") as file:
    data = file.read()
```

```
# Python code to illustrate split() function

with open("file.text", "r") as file:
    data = file.readlines()
    for line in data:
        word = line.split()
        print (word)
```

seek() method

In Python, seek() function is used to change the position of the File Handle to a given specific position. File handle is like a cursor, which defines from where the data has to be read or written in the file.

Syntax:

f.seek(offset, from_what), where f is file pointer

Parameters:

Offset: Number of positions to move forward

from_what: It defines point of reference.

Returns: Return the new absolute position.

Change the current file position to 4, (from beggining and return the rest of the line):

```
f = open("kk.txt", "r")
f.seek(4)   OR f.seek(4,0)
print(f.read())
```

Change the current file position to 4, from end, and return the rest of the line):

```
f = open("kk.txt", "r")
f.seek(-4,2)
print(f.read())
```

Common list of methods in txt mode

Method	Description
`close()`	Closes an opened file
`read(n)`	Reads the file content
`seek(offset,from=SEEK_SET)`	Changes the file position to `offset` bytes, in reference to `from` (start, current, end)
`tell()`	Returns an integer that represents the current position of the file's object
`write(s)`	Writes a string to the file
`writelines(lines)`	Writes a list of `lines` to the file

Regular Expressions:

A **Regular Expressions (RegEx)** is a special sequence of characters that uses a search pattern to find a string or set of strings.

It can detect the presence or absence of a text by matching with a particular pattern, and also can split a pattern into one or more sub-patterns.

Python provides a **re** module that supports the use of regex in Python. Its primary function is to offer a search, where it takes a regular expression and a string.

Here, it either returns the first match or else none

```
import re

#Check if the string starts with "The" and ends with "Spain":

txt = "The rain in Spain"
x = re.search("^The.*Spain$", txt)
if x:
        print("YES! We have a match!")
else:
        print("No match")
```

Examlpe:1

```
import re
s = ' A computer science portal for Data science using python'
match = re.search(r'portal', s)
print('Start Index:', match.start())
print('End Index:', match.end())
```

The above code gives the starting index and the ending index of the string portal.

Note: Here r character (r'portal') stands for raw, not regex. The raw string is slightly different from a regular string, it won't interpret the \ character as an escape character. This is because the regular expression engine uses \ character for its own escaping purpose.
Before starting with the Python regex module let's see how to actually write regex using metacharacters or special sequences.

RegEx Functions:

The re module offers a set of functions that allows us to search a string for a match:

Function	Description
Findall()	- Returns a list containing all matches
Search()	- Returns a Match object if there is a match anywhere in the string
Match()	- Returns a Match object found at begining of the string (at start index)
Split()	- Returns a list where the string has been split at each match
sub ()	- Replaces one or many matches with a string

The search() Function:

The search() function searches the string for a match, and returns a Match object if there is a match. If there is more than one match, only the first occurrence of the match will be returned:

```
import re
txt = "The rain in Spain"
x = re.search("\s", txt)
print("The first white-space character is located in position:", x.start())
```

```
import re
txt = "The rain in Spain"
x = re.search("Portugal", txt)
print(x)
```

The split() Function:

The split() function returns a list where the string has been split at each match:

```
import re
#Split the string at every white-space character:
txt = "The rain in Spain"
x = re.split("\s", txt)
print(x)
```

```
import re
#Split the string at the first white-space character:
txt = "The rain in Spain"
x = re.split("\s", txt, 1)
print(x)
```

The sub() Function:

The sub () function replaces the matches with the text of your choice:

```
import re
#Replace all white-space characters with the digit "9":
txt = "The rain in Spain"
x = re.sub("\s", "9", txt)
print(x)
```

Match Object:

A Match Object is an object containing information about the search and the result.

```
import re
#The search() function returns a Match object:
txt = "The rain in Spain"
x = re.search("ai", txt)
print(x)
```

Special Sequence:

A special sequence is a \ followed by one of the characters in the list below, and has a special meaning:

Character Description

\A - Returns a match if the specified characters are at the beginning of the string

Example : "\AThe"

\b - Returns a match where the specified characters are at the beginning or at the end of a word (the "r" in the beginning is making sure that the string is being treated as a "raw string")

Example: r"\bain" r"ain\b"

\B - Returns a match where the specified characters are present, but NOT at the beginning (or at the end) of a word (the "r" in the beginning is making sure that the string is being treated as a "raw string")

Example: r"\Bain" r"ain\B"

\d - Returns a match where the string contains digits (numbers from 0-9)

Example: "\d"

\D - Returns a match where the string DOES NOT contain digits

Example: "\D"

\s - Returns a match where the string contains a white space character

Example: "\s"

\S - Returns a match where the string DOES NOT contain a white space character

Example: "\S"

\w - Returns a match where the string contains any word characters (characters from a to Z, digits from 0-9, and the underscore _ character)

Example: "\w"

\W - Returns a match where the string DOES NOT contain any word characters

Example: "\W"

\Z - Returns a match if the specified characters are at the end of the string

Example: "Spain\Z"

Example1:

```
import re
txt = "The rain in Spain"

#Check if the string starts with "The":
x = re.findall("\AThe", txt)
print(x)
if x:
        print("Yes, there is a match!")
else:
        print("No match")
```

Example 2:

```
import re
txt = "The rain in Spain"
#Check if "ain" is present at the beginning of a WORD:
x = re.findall(r"\bain", txt)
print(x)
if x:
        print("Yes, there is at least one match!")
else:
        print("No match")
```

Example 3:

```
import re
txt = "The rain in Spain"
#Check if "ain" is present, but NOT at the beginning of a word:
x = re.findall(r"\Bain", txt)
print(x)
if x:
        print("Yes, there is at least one match!")
else:
        print("No match")
```

Example 4:

```
import re
txt = "The rain in Spain"
#Check if the string contains any digits (numbers from 0-9):
x = re.findall("\d", txt)
print(x)
if x:
```

```
    print("Yes, there is at least one match!")
else:
    print("No match")
```

Example 5:

```
import re
txt = "The rain in Spain"

#Return a match at every word character (characters from a to Z, digits from 0-9,
and the  underscore _ character):

x = re.findall("\w", txt)
print(x)
if x:
    print("Yes, there is at least one match!")
else:
    print("No match")
```

Metacharacters:

Metacharacters are characters with a special meaning:

Character	Description	Example
[]	A set of characters	"[a-m]"
\	Signals a special sequence (can also be used to escape special characters)	"\d"
.	Any character (except newline character)	"he..o"
^	Starts with	"^hello"
$	Ends with	"planet$"
*	Zero or more occurrences	"he.*o"
+	One or more occurrences	"he.+o"
?	Zero or one occurrences	"he.?o"
{}	Exactly the specified number of occurrences	"he.{2}o"
|	Either or	"falls|stays

\ – Backslash

The backslash (\) makes sure that the character is not treated in a special way. This can be considered a way of escaping metacharacters. For example, if you want to search for the dot(.) in the string then you will find that dot(.) will be treated as a special character as is one of the metacharacters (as shown in the above table). So for this case,

we will use the backslash(\) just before the dot(.) so that it will lose its specialty.

See the below example for a better understanding.

Example:

```
import re
s = ' A computer science portal for Data science using python'
# without using \
match = re.search(r'.', s)
print(match)
# using \
match = re.search(r'\.', s)
print(match)
```

[] – Square Brackets

Square Brackets ([]) represents a character class consisting of a set of characters that we wish to match.
For example, the character class [abc] will match any single a, b, or c.

We can also specify a range of characters using – inside the square brackets.
For example,

- [0, 3] is sample as [0123]
- [a-c] is same as [abc]

We can also invert the character class using the caret(^) symbol.

For example,

- [^0-3] means any number except 0, 1, 2, or 3
- [^a-c] means any character except a, b, or c

^ – Caret

Caret (^) symbol matches the beginning of the string i.e. checks whether the string starts with the given character(s) or not.

^g will check if the string starts with g such as geeks, globe, girl, g, etc.

•^gr will check if the string starts with ge such as greets, greetingsforgreetings, etc.

$ – Dollar

Dollar($) symbol matches the end of the string i.e checks whether the string ends with the given character(s) or not. For example –

•s$ will check for the string that ends with a such as geeks, ends, s, etc.

•ks$ will check for the string that ends with ks such as geeks, geeksforgeeks, ks, etc.

. – Dot

Dot(.) symbol matches only a single character except for the newline character (\n). For example –

- a.b will check for the string that contains any character at the place of the dot such as acb, acbd, abbb, etc
- .. will check if the string contains at least 2 characters

| – Or

Or symbol works as the or operator meaning it checks whether the pattern before or after the or symbol is present in the string or not.

For example –

- a|b will match any string that contains a or b such as acd, bcd, abcd, etc.

? – Question Mark

Question mark(?) checks if the string before the question mark in the regex occurs at least once or not at all.

For example –

•ab?c will be matched for the string ac, acb, dabc but will not be matched for abbc because there are two b. Similarly, it will not be matched for abdc because b is not followed by c.

* – Star

Star (*) symbol matches zero or more occurrences of the regex preceding the * symbol.

For example –

•ab*c will be matched for the string ac, abc, abbbc, dabc, etc.

but will not be matched for abdc because b is not followed by c.

+ – Plus

Plus (+) symbol matches one or more occurrences of the regex preceding the + symbol.

For example –

•ab+c will be matched for the string abc, abbc, dabc, but will not be matched for ac, abdc because there is no b in ac and b is not followed by c in abdc.

{m, n} – Braces

Braces match any repetitions preceding regex from m to n both inclusive.

For example –

•a{2, 4} will be matched for the string aaab, baaaac, gaad, but will not be matched for strings like abc, bc because there is only one a or no a in both the cases.

(<regex>) – Group

Group symbol is used to group sub-patterns.

For example –

•(a|b)cd will match for strings like acd, abcd, gacd, etc.

Example 1:

```
import re
txt = "The rain in Spain"
#Find all lower case characters alphabetically between "a" and "m":
x = re.findall("[a-m]", txt)
print(x)
```

Example 2

```
import re
txt = "hell planet"
#Search for a sequence that starts with "he", followed by 1 or more (any)
characters, and an
"o":
x = re.findall("he.+o", txt)
print(x)
if x:
        print("match found")
else:
        print("no match")
```

Example 3:

```
import re
txt = "hell planet"
#Search for a sequence that starts with "he", followed by 1 or more (any) characters,
and an
"o":
x = re.findall("he.{2}o", txt)
print(x)
if x:
        print("match found")
else:
        print("no match")
```

Example 4

```
import re

txt = "hello planet"

#Check if the string starts with 'hello':

x = re.findall("^hello", txt)

if x:

print("Yes, the string starts with 'hello'")

else:

print("No match")
```

Example 5

```
import re
```

```
txt = "The rain in Spain falls mainly in the plain!"
#Check if the string contains either "falls" or "stays":
x = re.findall("foailil|stays", txt)
print(x)
if x:
        print("Yes, there is at least one match!")
else:
        print("No match")
```

Sets:

A set is a set of characters inside a pair of square brackets [] with a special meaning:

Set	Description
[arn]	Returns a match where one of the specified characters (a, r, or n) are present
[a-n]	Returns a match for any lower case character, alphabetically between a and n
[^arn]	Returns a match for any character EXCEPT a, r, and n
[0123]	Returns a match where any of the specified digits (0, 1, 2, or 3) are present
[0-9]	Returns a match for any digit between 0 and 9
[0-5][0-9]	Returns a match for any two-digit numbers from 00 and 59
[a-zA-Z]	Returns a match for any character alphabetically between a and z, lower case OR upper case

In sets, +, *, ., |, (), $,{} has no special meaning, so [+] means: return a match for any + character in the string

Example:

```
import re
txt = "The rain in Spain"
#Check if the string has any a, r, or n characters:
x = re.findall("[arn]", txt)
print(x)
if x:
        print("Yes, there is at least one match!")
else:
        print("No match")
```

Example:

```
import re
txt = "8 times before 11:45 AM"
#Check if the string has any digits:
x = re.findall("[0-9]", txt)
print(x)
if x:
        print("Yes, there is at least one match!")
else:
        print("No match")
```

Example:

```
import re
txt = "8 times before 11:45 AM"
#Check if the string has any two-digit numbers, from 00 to 59:
x = re.findall("[0-5][0-9]", txt)
print(x)
if x:
        print("Yes, there is at least one match!")
else:
        print("No match")
```

Pandas in Python for Data Science

Introduction to Pandas

Pandas is an open-source Python library for data manipulation and analysis. It provides fast, flexible, and expressive data structures designed to work with structured (tables, time-series) and semi-structured data.

Pandas gives you answers about the data. Like:

- Is there a correlation between two or more columns?
- What is average value?
- Max value?
- Min value?

Pandas are also able to delete rows that are not relevant, or contains wrong values, like empty or NULL values. This is called cleaning the data.

Pandas is widely used in **Data Science**, **Machine Learning**, and **Data Engineering** to clean, transform, and analyze data efficiently.

Why Use Pandas?

- Handles large datasets efficiently
- Provides powerful functions for data analysis
- Integrates well with NumPy, Matplotlib, and Scikit-learn
- Offers built-in functions for reading/writing files (CSV, Excel, SQL, JSON, etc.)

Installation

If you haven't installed Pandas, you can do so using:

If you have Python and PIP already installed on a system, then

pip install pandas

C:\users\desktop\pip install pandas

Import Pandas

Once Pandas is installed, import it in your applications by adding the import keyword:

```
import pandas as pd
```

Pandas Version Checking:

```
import pandas as pd

print(pd.__version__)
```

1. Pandas Data Structures

Pandas provides two primary data structures:

- **Series** (1D labeled array)
- **DataFrame** (2D table-like structure)

1.1 Series (One-Dimensional Data)

A Series is similar to a **column in a spreadsheet** or **an array in NumPy**. It consists of values and an associated index.

Creating a Pandas Series

```
import pandas as pd

data = [10, 20, 30, 40, 50]
series = pd.Series(data)
print(series)
```

Output:

```
0    10
1    20
2    30
3    40
4    50
dtype: int64
```

Custom Indexing

```
data = [10, 20, 30, 40, 50]

series = pd.Series(data, index=['A', 'B', 'C', 'D', 'E'])
print(series)
```

Output:

```
A    10
B    20
```

```
C    30
D    40
E    50
dtype: int64
```

Accessing Elements

```
print(series['C'])

# Output: 30

print(series[2])
# Output: 30
```

Key/Value Objects as Series

Create a simple Pandas Series from a dictionary:

```
import pandas as pd

data = {'A' :10,'B': 20,'C': 30,'D': 40, 'E': 50}

series = pd.Series(data)
print(series)
```

Output:

```
A    10
B    20
C    30
D    40
E    50
dtype: int64
```

1.2 DataFrame (Two-Dimensional Data)

A DataFrame is similar to an **Excel spreadsheet or SQL table**. It consists of multiple columns, each containing a Series.

Data sets in Pandas are usually multi-dimensional tables, called DataFrames.

Series is like a column, a DataFrame is the whole table.

Creating a DataFrame from a Dictionary

```
data = {
   'Name': ['Alice', 'Bob', 'Charlie'],
   'Age': [25, 30, 35],
   'Salary': [50000, 60000, 70000]
```

```
}

df = pd.DataFrame(data)
print(df)
```

Output:

```
    Name  Age  Salary
0   Alice   25  50000
1     Bob   30  60000
2  Charlie   35  70000
```

Creating a DataFrame from a CSV File

```
# Load data from CSV

df = pd.read_csv("data.csv")

# Display first 5 rows

print(df.head())
```

2. Basic DataFrame Operations

2.1 Viewing Data

```
print(df.head())
# First 5 rows
print(df.tail())
# Last 5 rows
print(df.shape)
 # (rows, columns) - No of rows and columns
print(df.info())
# Summary of DataFrame - No of columns and cloumn names and dat type
print(df.describe())
# Summary statistics   - It display count, mean, std, min, 25%, 50%, 75% and Max
```

2.2 Selecting Data

Selecting a Single Column

```
print(df['Name'])
```

Selecting Multiple Columns

```
print(df[['Name', 'Salary']])
```

Selecting Rows by Index

```
print(df.iloc[1])  # Select second row
print(df.loc[1])   # Select row with index 1
Print(df.loc[0,1]  # select list of index
```

2.3 Filtering Data

Using Conditions

```
filtered_df = df[df['Age'] > 25]  # Select rows where Age > 25
print(filtered_df)
```

Using Multiple Conditions

```
filtered_df = df[(df['Age'] > 25) & (df['Salary'] > 50000)]
print(filtered_df)
```

2.4 Modifying Data

Adding a New Column

```
df['Experience'] = [2, 5, 7]  # Adding a new column
print(df)
```

Modifying Column Values

```
df.loc[1, 'Salary'] = 65000  # Change salary for index 1
print(df)
```

Deleting a Column

```
df.drop(columns=['Experience'], inplace=True)
print(df)
```

3. Handling Missing Data

Check for Missing Values

```
print(df.isnull().sum())  # Count missing values in each column
```

Remove Missing Data

```
df.dropna(inplace=True)  # Remove rows with missing values
```

Fill Missing Data

```
df.fillna(0, inplace=True)  # Replace NaN with 0
```

4. Grouping and Aggregations

Grouping by a Column

```
df_grouped = df.groupby('Age').mean()  # Group by Age and calculate mean
print(df_grouped)
```

Applying Aggregations

```
print(df['Salary'].mean())  # Average salary
print(df['Salary'].sum())   # Total salary
print(df['Salary'].max())   # Maximum salary
print(df['Salary'].min())   # Minimum salary
```

5. Sorting Data

Sort by a Single Column

```
df_sorted = df.sort_values(by='Salary', ascending=False)  # Sort descending
print(df_sorted)
```

Sort by Multiple Columns

```
df_sorted = df.sort_values(by=['Age', 'Salary'], ascending=[True, False])
print(df_sorted)
```

6. Exporting Data

Save to CSV

```
df.to_csv("output.csv", index=False)
```

Save to Excel

```
df.to_excel("output.xlsx", index=False)
```

7. Working with Large Datasets

For large datasets, Pandas provides efficient methods:

- **Load data in chunks:**

```
chunk_size = 1000
for chunk in pd.read_csv("large_data.csv", chunksize=chunk_size):
    process(chunk)  # Perform operations on each chunk
```

- **Optimize data types:**

```
df['Salary'] = df['Salary'].astype('int32')  # Reduce memory usage
```

8. Working with Time Series Data

Pandas provides extensive support for time series data.

Convert a Column to DateTime

```
df['Date'] = pd.to_datetime(df['Date'])
```

Set a Column as Index

```
df.set_index('Date', inplace=True)
```

Resample Data (e.g., Monthly)

```
df_resampled = df.resample('M').mean()
```

Conclusion

Pandas is an essential library for **data analysis, data manipulation, and data science**. It simplifies working with structured data, making tasks like **data cleaning, transformation, and aggregation** easy.

Key Points:

- Pandas provides **Series** (1D) and **DataFrame** (2D) structures
- Supports **CSV, Excel, JSON, SQL**, and more file formats
- Efficiently handles **missing data, sorting, filtering, and grouping**
- Ideal for **EDA (Exploratory Data Analysis)**
- Optimized for **large datasets and time-series analysis**

Pandas Practical Project: Analyzing Sales Data

In this project, we will analyze **supermarket sales data** using Pandas.

The dataset contains information about customer purchases, including **date, branch, city, product category, unit price, quantity, total sales, payment method, and customer type**.

Steps:

- Load and inspect the dataset
- Clean and preprocess the data
- Perform exploratory data analysis (EDA)
- Generate insights using grouping, filtering, and visualization

1. Install and Import Required Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

2. Load the Dataset

We assume the dataset is in **CSV format** and named sales_data.csv.

```
df = pd.read_csv("sales_data.csv")
```

Preview the Data

```
print(df.head())  # Show first 5 rows
print(df.info())  # Get data types and missing values
```

3. Data Cleaning & Preprocessing

Check for Missing Values

```
print(df.isnull().sum())  # Count missing values
```

Drop Missing Values (if any)

```
df.dropna(inplace=True)
```

Convert Date Column to DateTime Format

```
df['Date'] = pd.to_datetime(df['Date'])
```

Check for Duplicate Records

```
print(df.duplicated().sum())  # Count duplicate rows
df.drop_duplicates(inplace=True)
```

4. Exploratory Data Analysis (EDA)

Basic Statistics

```
print(df.describe())  # Summary statistics for numerical columns
```

Top 5 Most Sold Products

```
top_products = df['Product'].value_counts().head(5)
print(top_products)
```

5. Sales Analysis

Total Revenue

```
total_revenue = df['Total'].sum()
print(f"Total Revenue: ${total_revenue}")
```

Revenue Per City

```
revenue_per_city = df.groupby("City")["Total"].sum()
print(revenue_per_city)
```

Revenue Per Product Category

```
revenue_per_category = df.groupby("Category")["Total"].sum()
print(revenue_per_category)
```

6. Data Visualization

1. Sales Trend Over Time

```
df.groupby(df['Date'].dt.month)['Total'].sum().plot(kind='line', marker='o',
figsize=(10,5), title="Sales Trend Over Months")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.show()
```

2. Most Popular Payment Methods

```
plt.figure(figsize=(8,5))
sns.countplot(x='Payment Method', data=df, palette='pastel')
plt.title("Most Used Payment Methods")
```

```
plt.show()
```

3. Top Selling Products

```
plt.figure(figsize=(10,5))
df['Product'].value_counts().head(10).plot(kind='bar', color='skyblue')
plt.title("Top 10 Best-Selling Products")
plt.xlabel("Product Name")
plt.ylabel("Number of Sales")
plt.xticks(rotation=45)
plt.show()
```

4. Revenue by City

```
plt.figure(figsize=(8,5))
sns.barplot(x=revenue_per_city.index, y=revenue_per_city.values, palette="viridis")
plt.title("Revenue by City")
plt.ylabel("Total Sales")
plt.show()
```

Conclusion and Insights

Which city generates the most revenue?

Which product category is the most profitable?
What is the trend of sales over time?
Which payment method is preferred by customers?

Further Improvements

Apply **Machine Learning** to predict future sales
Perform **Customer Segmentation** using clustering techniques
Build an **interactive dashboard** using Streamlit

NumPy for Data Science in Python

Introduction to NumPy

NumPy (**Numerical Python**) is a fundamental Python library for scientific computing. It provides support for large, multi-dimensional arrays and matrices, along with mathematical functions to perform operations on these data structures.

- Efficient storage & operations on large datasets
- Faster than Python lists (because of optimized C-based implementation)
- Essential for Machine Learning & Data Science
- Works seamlessly with Pandas, SciPy, Matplotlib, and Scikit-learn

Installing NumPy

If you haven't installed NumPy, install it using:

```
pip install numpy
```

Importing NumPy

```
import numpy as np
```

1. NumPy Arrays (ndarray)

The core object in NumPy is the ndarray (n-dimensional array).

It is **faster, more memory-efficient, and supports vectorized operations**, unlike Python lists.

Creating NumPy Arrays

a) From a Python List

```
arr = np.array([1, 2, 3, 4, 5])
print(arr)
print(type(arr))

# Output: <class 'numpy.ndarray'>
```

b) Multi-Dimensional Array

```
arr2D = np.array([[1, 2, 3], [4, 5, 6]])
print(arr2D)
```

2. Array Properties

```
print(arr.ndim)  # Number of dimensions (1D, 2D, etc.)
print(arr.shape)  # Shape (rows, columns)
print(arr.size)  # Total elements
print(arr.dtype)  # Data type of elements
```

3. Creating Special Arrays

NumPy provides built-in functions to create arrays quickly.

```
np.zeros((3, 4))  # 3x4 array filled with 0s
np.ones((2, 3))  # 2x3 array filled with 1s
np.full((3, 3), 7)  # 3x3 array filled with 7s
np.eye(4)  # 4x4 Identity matrix
```

Random Arrays

```
np.random.rand(3, 3)  # Random values between 0 and 1
np.random.randint(10, 50, (3, 3))  # Random integers from 10 to 50
```

4. Indexing & Slicing in NumPy

Accessing Elements

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[2])

# Output: 30
```

Accessing 2D Arrays

```
arr2D = np.array([[10, 20, 30], [40, 50, 60]])
print(arr2D[1, 2])

# Row 1, Column 2 → Output: 60
```

Slicing Arrays

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[1:4])

# Output: [20 30 40]
```

5. Mathematical Operations on Arrays

NumPy allows **vectorized operations**, making mathematical computations **faster and more efficient**.

Element-wise Operations

```
arr = np.array([1, 2, 3, 4, 5])
print(arr + 10)  # Add 10 to each element
print(arr * 2)   # Multiply each element by 2
print(arr ** 2)  # Square each element
```

Universal Functions (ufunc)

```
np.sqrt(arr)  # Square root
np.exp(arr)  # Exponential function
np.log(arr)  # Natural logarithm
np.sin(arr)  # Sine function
np.mean(arr)  # Mean (average)
np.median(arr)  # Median
np.std(arr)  # Standard deviation
np.sum(arr)  # Sum of all elements
np.max(arr)  # Maximum value
np.min(arr)  # Minimum value
```

6. Reshaping and Transposing

Reshaping Arrays

```
arr = np.array([1, 2, 3, 4, 5, 6])
reshaped = arr.reshape(2, 3)  # Convert 1D → 2D
print(reshaped)
```

Transposing Arrays

```
arr2D = np.array([[1, 2, 3], [4, 5, 6]])
print(arr2D.T)

 # Swap rows & columns
```

7. Concatenation and Stacking

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

np.concatenate((arr1, arr2))

 # Combine arrays

np.vstack((arr1, arr2))
```

```
# Stack vertically
np.hstack((arr1, arr2))
 # Stack horizontally
```

8. Boolean Masking & Filtering

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[arr > 20])

# Filter values greater than 20
```

9. NumPy for Data Science

NumPy is widely used in **data science, machine learning, and AI**.

Loading Data from Files

```
data = np.loadtxt('data.csv', delimiter=',', skiprows=1)
print(data)
```

Simulating Data for Machine Learning

```
X = np.random.rand(100, 3)
 # 100 samples, 3 features
y = np.random.randint(0, 2, 100)
# Binary labels (0 or 1)
```

Practical NumPy Project

Analyzing Sales Data using NumPy

Simulate a Dataset

```
np.random.seed(42)  # Ensure reproducibility

# Generate sales data (Product Price, Quantity Sold, Revenue)
prices = np.random.randint(100, 500, 10)
quantities = np.random.randint(1, 20, 10)
revenues = prices * quantities  # Total Revenue per Product

sales_data = np.column_stack((prices, quantities, revenues))

print("Price | Quantity | Revenue")
print(sales_data)
```

Calculate Insights

```
print("Total Revenue:", np.sum(revenues))
print("Average Revenue per Product:", np.mean(revenues))
print("Max Revenue Product:", np.max(revenues))
print("Min Revenue Product:", np.min(revenues))
```

Find Best-Selling Products

```
top_3 = np.argsort(revenues)[-3:]  # Get indices of top 3 products
print("Top 3 Products:", sales_data[top_3])
```

NumPy is the backbone of **scientific computing, machine learning, and data science**. It provides **efficient numerical operations, large-scale data processing, and seamless integration** with other libraries like **Pandas, SciPy, and TensorFlow**.

Key points:

- Faster than Python lists
- Supports multi-dimensional arrays (ndarray)
- Efficient mathematical operations & aggregations
- Useful for data science & machine learning

NumPy + Pandas Data Science Project: Analyzing E-commerce Sales Data

Objective:

We will analyze a dataset containing **e-commerce sales transactions**, including **order dates, product categories, prices, customer locations, and payment methods**. We will leverage **NumPy for numerical operations** and **Pandas for data manipulation**.

1. Install & Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

2. Load Dataset (CSV File)

Assuming we have a dataset ecommerce_sales.csv with the following columns:

- **Order ID** – Unique transaction ID

- **Order Date** – Date of purchase
- **Category** – Product category
- **Price** – Price per unit
- **Quantity** – Number of items purchased
- **Total Sales** – Revenue generated
- **Customer Location** – Country/City
- **Payment Method** – Payment type (Credit Card, PayPal, etc.)

```
df = pd.read_csv("ecommerce_sales.csv")
print(df.head())
# View first 5 rows
print(df.info())
# Check data types and missing values
```

3. Data Preprocessing & Cleaning

Convert Date Column to DateTime Format

```
df["Order Date"] = pd.to_datetime(df["Order Date"])
```

Handle Missing Values

```
df.dropna(inplace=True)  # Remove rows with missing values
```

Remove Duplicate Records

```
df.drop_duplicates(inplace=True)
```

Ensure Correct Data Types

```
df["Price"] = df["Price"].astype(float)
df["Quantity"] = df["Quantity"].astype(int)
df["Total Sales"] = df["Price"] * df["Quantity"]
```

4. Exploratory Data Analysis (EDA)

Basic Statistics

```
print(df.describe())  # Summary statistics for numerical columns
print(df["Category"].value_counts())  # Count of each product category
```

Total Revenue Calculation

```
total_revenue = np.sum(df["Total Sales"])
```

```
print(f"Total Revenue: ${total_revenue}")
```

Revenue Per Product Category

```
revenue_by_categoy = df.groupby("Category")["Total Sales"].sum()
print(revenue_by_category)
```

5. Data Visualization

1. Sales Trend Over Time

```
df.groupby(df["Order Date"].dt.month)["TotalSales"].sum().plot(kind="line", marker="o", figsize=(10,5), title="Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.show()
```

2. Best-Selling Product Categories

```
plt.figure(figsize=(10,5))
df["Category"].value_counts().plot(kind="bar", color="skyblue")
plt.title("Top Selling Product Categories")
plt.xlabel("Category")
plt.ylabel("Number of Sales")
plt.xticks(rotation=45)
plt.show()
```

3. Revenue Distribution Across Locations

```
plt.figure(figsize=(8,5))
sns.barplot(x=revenue_by_category.index, y=revenue_by_category.values, palette="viridis")
plt.title("Revenue by Product Category")
plt.ylabel("Total Revenue")
plt.xticks(rotation=45)
plt.show()
```

4. Most Popular Payment Methods

```
plt.figure(figsize=(8,5))
sns.countplot(x="Payment Method", data=df, palette="pastel")
plt.title("Most Used Payment Methods")
plt.show()
```

6. Advanced Insights Using NumPy

Top 5 Orders with Highest Sales

```
top_orders = df.nlargest(5, "Total Sales")
print(top_orders)
```

Find the Most Profitable Product

```
most_profitable = df.groupby("Category")["Total Sales"].sum().idxmax()
print(f"The most profitable product category is: {most_profitable}")
```

Sales Performance by Month

```
monthly_sales = df.groupby(df["Order Date"].dt.month)["Total
Sales"].sum().to_numpy()
print("Monthly Sales:", monthly_sales)
```

7. Machine Learning Integration (Optional)

We can use **NumPy and Pandas** to prepare data for **predictive modeling** using **Scikit-Learn**.

For example, we can predict **future sales trends** using **linear regression**.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df["Month"] = df["Order Date"].dt.month  # Extract month
X = df[["Month"]]  # Feature
y = df["Total Sales"]  # Target variable
```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```
model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

8. Conclusion & Insights

What is the best-selling product category?

Which payment method is the most popular?

How do sales fluctuate over months?

Which location contributes the most revenue?

Can we predict future sales using machine learning?

Next Steps:

- Integrate this analysis into a **Streamlit Dashboard**
- Use **Matplotlib & Seaborn** for **more interactive visualizations**
- Deploy a **machine learning model** to predict future sales

Regular Expressions

Regular Expressions in Python

What are Regular Expressions?

Regular Expressions (**regex**) are powerful tools used to **search, match, and manipulate** text using patterns. They are widely used in **data validation, text processing, web scraping, and data cleaning**.

Python provides regex functionality through the built-in re module.

1. Importing the re Module

To work with regular expressions in Python, we must import the re module:

```
import re
```

2. Basic Regex Functions in Python

Function	Description
re.search()	Searches for the first occurrence of a pattern in a string.

Function	Description
re.match()	Checks if a pattern matches the **beginning** of a string.
re.findall()	Returns **all occurrences** of a pattern in a string.
re.finditer()	Returns an iterator with **match objects** for all occurrences.
re.sub()	Replaces occurrences of a pattern with a replacement string.
re.split()	Splits a string based on a pattern.

---

3. Regex Metacharacters

Symbol	Description	Example
.	Matches any **single character** except newline	r"a.b" → Matches "acb", "axb"
^	Matches **beginning** of a string	r"^Hello" → "Hello World" ✅, "Hi Hello" ❌
$	Matches **end** of a string	r"World$" → "Hello World" ✅
*	Matches **0 or more** occurrences	r"ab*" → "a", "ab", "abb"
+	Matches **1 or more** occurrences	r"ab+" → "ab", "abb" but not "a"
?	Matches **0 or 1** occurrence	r"colou?r" → "color", "colour"
{m,n}	Matches **m to n** occurrences	r"a{2,4}" → "aa", "aaa", "aaaa"
[]	Matches any **one character** in brackets	r"[aeiou]" → "a", "e", "i"
\d	Matches any **digit** (0-9)	r"\d+" → "123", "456"
\w	Matches any **word character** (a-z, A-Z, 0-9, _)	r"\w+" → "word", "Python3"
\s	Matches any **whitespace** character	r"\s+" → " "
`	`	OR operator

---

4. Common Regex Use Cases

1. Check if a string contains a word

```
import re
text = "Python is a great programming language"
match = re.search(r"Python", text)
if match:
    print("Match found!")

# Output: Match found!
```

2. Extract all email addresses from text

```
text = "Contact us at support@example.com or sales@example.com"
emails = re.findall(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}", text)
```

```
print(emails)

 # Output: ['support@example.com', 'sales@example.com']
```

3. Validate a phone number (US Format)

```
pattern = r"\(\d{3}\) \d{3}-\d{4}"
phone = "(123) 456-7890"
if re.fullmatch(pattern, phone):
    print("Valid phone number!")

# Output: Valid phone number!
```

4. Extract all numbers from a string

```
text = "The price is 100 dollars and 50 cents."
numbers = re.findall(r"\d+", text)
print(numbers)

# Output: ['100', '50']
```

5. Replace Multiple Spaces with a Single Space

```
text = "Python    is    awesome!"
clean_text = re.sub(r"\s+", " ", text)
print(clean_text)

#Output: "Python is awesome!"
```

5. Advanced Regex Techniques

1. Using re.compile() for Reusability

Instead of writing the same pattern multiple times, use re.compile()

```
pattern = re.compile(r"\d{4}")  # Matches any 4-digit number
matches = pattern.findall("Year: 2021, Code: 1234, ID: 5678")
print(matches)

 # Output: ['2021', '1234', '5678']
```

2. Extracting HTML Tags (Web Scraping)

```
html = "<html><head><title>My
Website</title></head><body>Content</body></html>"
```

```
tags = re.findall(r"<.*?>", html)
print(tags)

Output: ['<html>', '<head>', '<title>', '</title>', '</head>', '<body>', '</body>','</html>']
```

3. Extracting Hashtags from Tweets

```
tweet = "Learning #Python is fun! #Coding #100DaysOfCode"
hashtags = re.findall(r"#\w+", tweet)
print(hashtags)  # ✅ Output: ['#Python', '#Coding', '#100DaysOfCode']
```

4. Extracting Dates from Text

```
text = "Today's date is 2024-01-26 and yesterday was 2024-01-25."
dates = re.findall(r"\d{4}-\d{2}-\d{2}", text)
print(dates)

# Output: ['2024-01-26', '2024-01-25']
```

5. Checking for Strong Passwords

```
password = "P@ssw0rd123"
pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-
z\d@$!%*?&]{8,}$"

if re.fullmatch(pattern, password):
    print("Strong password!")  # ✅ Output: Strong password!
```

Explanation:

- (?=.*[A-Z]) → At least **one uppercase letter**
- (?=.*[a-z]) → At least **one lowercase letter**
- (?=.*\d) → At least **one digit**
- (?=.*[@$!%*?&]) → At least **one special character**
- {8,} → Minimum **8 characters**

6. Conclusion

- Regular expressions are **powerful tools** for text processing.
- **Use** re.search()**,** re.findall()**,** re.sub()**, and** re.split() for efficient text matching and manipulation.
- **Combine regex with web scraping, data validation, and log file analysis** for real-world applications.

Web Scraping in Python

Web Scraping is the process of extracting **data from websites** using automated scripts. It is widely used in **data science, market research, competitive analysis, and automation**.

Basics of web scraping
Using requests and BeautifulSoup
Extracting data from websites
Scraping dynamic websites with Selenium
Saving data to CSV/Excel
Handling anti-scraping measures

1. Prerequisites

To start web scraping, install the required libraries:

pip install requests beautifulsoup4 lxml selenium pandas

2. How Web Scraping Works

Web pages are built using **HTML** and structured with **CSS**. Web scraping involves:
Sending a request to the website
Extracting the HTML content
Parsing the HTML to find relevant data
Saving the data for further analysis

3. Basic Web Scraping with requests and BeautifulSoup

Fetch a Web Page

```
import requests

url = "https://example.com"
response = requests.get(url)

if response.status_code == 200:
print(response.text[:500])  # Print first 500 characters of the page
```

requests.get(url): Sends an HTTP request
.text: Returns the HTML content of the page

Parsing HTML with BeautifulSoup

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(response.text, "html.parser")
print(soup.prettify()[:500])
  # Display formatted HTML
```

4. Extracting Data from HTML

1. Extracting Titles & Paragraphs

```
title = soup.title.text
paragraphs = [p.text for p in soup.find_all("p")]

print("Title:", title)
print("Paragraphs:", paragraphs)
```

2. Extracting Links

```
links = [a["href"] for a in soup.find_all("a", href=True)]
print("Links:", links)
```

3. Extracting Images

```
images = [img["src"] for img in soup.find_all("img", src=True)]
print("Images:", images)
```

5. Scraping Dynamic Websites with Selenium

Some websites load content dynamically using **JavaScript**, which requests cannot handle. **Selenium** helps interact with such websites.

Install Selenium WebDriver

```
pip install selenium
```

Download Chrome WebDriver from: https://chromedriver.chromium.org/

Automating a Browser with Selenium

```
from selenium import webdriver

driver = webdriver.Chrome()  # Initialize WebDriver
driver.get("https://example.com")  # Open website
print(driver.title)  # Print page title
```

```
driver.quit()  # Close the browser
```

Extracting Data with Selenium

```
from selenium.webdriver.common.by import By
driver.get("https://example.com")
elements = driver.find_elements(By.TAG_NAME, "p")
for element in elements:
 print(element.text)

driver.quit()
```

6. Storing Scraped Data

Save to CSV

```
import pandas as pd

data = {"Title": [title], "Links": links, "Images": images}
df = pd.DataFrame(data)

df.to_csv("scraped_data.csv", index=False)
print("Data saved to CSV")
```

Save to Excel

```
df.to_excel("scraped_data.xlsx", index=False)
```

7. Handling Anti-Scraping Measures

Many websites implement **anti-scraping protections**,

such as:

Blocking frequent requests
Requiring JavaScript execution
CAPTCHAs and login requirements

Ways to Bypass Protections

Use headers & user-agents
Implement delays between requests
Use proxy servers

```
headers = {"User-Agent": "Mozilla/5.0"}
response = requests.get(url, headers=headers)
```

8. Real-World Web Scraping Projects

Scraping e-commerce prices (Amazon, eBay)
Scraping job listings (LinkedIn, Indeed)

Scraping news articles
Scraping social media posts

9. Ethical Considerations

Always follow **legal guidelines** when scraping websites:

- **Check the website's** robots.txt **file**
- **Do not overload the server with frequent requests**
- **Respect terms of service**

Example:

- https://example.com/robots.txt